

Natural Evolution Strategies (NES)

Chiến lược Tiến hóa tự nhiên

Lê Thiên Quân - 24520027
Đinh Mạnh Hùng - 24520014

Khoa Khoa học Máy tính
Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM

Ngày 5 tháng 1 năm 2026



Nội dung trình bày

- 1 Chương 1: Giới thiệu
- 2 Chương 2: Cơ sở toán học
- 3 Chương 3: xNES - Exponential Natural Evolution Strategies
- 4 Chương 4: So sánh & Kết luận



Chương 1:

Giới thiệu & Bối cảnh



1.1. Bài toán Tối ưu hóa hộp đen (Black-Box Optimization)

Thực trạng: Nhiều bài toán thực tế (y tế, hàng không, điều khiển tự động...) quá phức tạp để mô hình hóa trực tiếp.

Đặc điểm "Hộp đen":

- Không có thông tin về cấu trúc hàm mục tiêu $f(x)$.
- Không có đạo hàm giải tích.
- Chỉ có thể đánh giá $f(x)$ tại các điểm tham số cụ thể.

Mục tiêu

Tìm cực trị toàn cục mà không cần hiểu "bên trong" hàm số hoạt động như thế nào, thường dựa trên các phương pháp heuristic.



1.2. Lược sử các phương pháp tiếp cận

Bài toán này đã được nghiên cứu hơn 50 năm qua với nhiều hướng tiếp cận:

- **Cổ điển:** Nelder-Mead (Simplex), Quasi-Newton.
- **Lấy cảm hứng sinh học/vật lý:**
 - Simulated Annealing (Luyện kim).
 - Genetic Algorithms (Di truyền học).
 - Particle Swarm Optimization (Bầy đàn).
- **Evolution Strategies (ES):** Phát triển bởi Rechenberg & Schwefel (thập niên 60-70). Dùng đột biến (mutation) và chọn lọc (selection) trên các số thực.



1.3. Từ Heuristics đến Toán học chặt chẽ

CMA-ES (Covariance Matrix Adaptation - ES):

- Là chuẩn mực vàng của dòng ES.
- Sử dụng ma trận hiệp phương sai để mô hình hóa sự phụ thuộc giữa các biến.
- *Tuy nhiên*: Dựa nhiều vào heuristics phức tạp, khó phân tích động lực học một cách chính xác.

Yêu cầu: Cần một phương pháp có hiệu năng tương đương CMA-ES nhưng có *nền tảng toán học minh bạch* thay vì các quy tắc tay (hand-crafted rules). → **Sự ra đời của Natural Evolution Strategies (NES).**



1.4. Ý tưởng cốt lõi của NES

Khác với các phương pháp ES truyền thống (thường chọn lọc cá thể rồi tạo quần thể mới), NES tiếp cận theo hướng **Gradient Ascent trên Phân phối**:

Cơ chế hoạt động:

- 1 Duy trì một phân phối tìm kiếm $\pi(z|\theta)$ (VD: Gaussian).
- 2 Cập nhật tham số θ (VD: μ, Σ) theo hướng tăng kỳ vọng hàm thích nghi.
- 3 Sử dụng **Natural Gradient** để đảm bảo sự ổn định.

Sự khác biệt

NES không tối ưu hóa điểm dữ liệu. NES tối ưu hóa **phân phối** sinh ra dữ liệu đó.



1.5. Tiếp

Để thực hiện ý tưởng trên, chúng ta cần trả lời 2 câu hỏi lớn:

- ❶ Làm thế nào để tính gradient của một "kỳ vọng" khi không biết hàm $f(x)$?
- ❷ Tại sao Gradient thông thường lại thất bại trong việc cập nhật tham số phân phối (đặc biệt là phương sai)?

Chương 2 sẽ đi sâu vào cơ sở lý thuyết giải quyết hai vấn đề này.



Chương 2:

Search Gradients & Natural Gradient



2.1. Định nghĩa Search Gradient

Để giải quyết bài toán hộp đen một cách toán học, NES tối ưu hóa **Kỳ vọng hàm thích nghi** (Expected Fitness) thay vì tối ưu trực tiếp $f(x)$.

Gọi θ là tham số của phân phối tìm kiếm $\pi(z|\theta)$. Hàm mục tiêu $J(\theta)$ được định nghĩa:

$$J(\theta) = \mathbb{E}_{\theta}[f(z)] = \int f(z)\pi(z|\theta)dz \quad (1)$$

Vấn đề: Chúng ta cần tìm $\nabla_{\theta}J(\theta)$ để thực hiện Gradient Ascent:

$$\theta \leftarrow \theta + \eta \nabla_{\theta}J(\theta)$$

Nhưng $f(z)$ không khả vi hoặc không biết công thức. Làm sao tính đạo hàm?



2.2. Log-Likelihood Trick

Đây là bước quan trọng nhất để chuyển đạo hàm vào trong phân phối xác suất.

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int f(z) \pi(z|\theta) dz \\&= \int f(z) \nabla_{\theta} \pi(z|\theta) dz \quad (\text{Đưa đạo hàm vào trong tích phân}) \\&= \int f(z) \frac{\nabla_{\theta} \pi(z|\theta)}{\pi(z|\theta)} \pi(z|\theta) dz \quad (\text{Nhân và chia cho } \pi) \\&= \int f(z) \nabla_{\theta} \log \pi(z|\theta) \pi(z|\theta) dz \quad (\text{Quy tắc đạo hàm log}) \\&= \mathbb{E}_{z \sim \pi} [f(z) \nabla_{\theta} \log \pi(z|\theta)]\end{aligned}\tag{2}$$

Kết quả

Ta chỉ cần tính đạo hàm của mật độ xác suất $\log \pi$ (thứ ta đã biết), không cần đạo hàm của $f(z)$ (thứ ta không biết).



2.3. Ước lượng Monte Carlo

Trong thực tế, ta không tính tích phân mà xấp xỉ bằng λ mẫu thử $z_1, \dots, z_\lambda$ được sinh từ $\pi(\cdot|\theta)$:

$$\nabla_\theta J(\theta) \approx \frac{1}{\lambda} \sum_{k=1}^{\lambda} f(z_k) \nabla_\theta \log \pi(z_k|\theta) \quad (3)$$

Đây chính là nền tảng của thuật toán **Canonical Search Gradient**.

- $\nabla_\theta \log \pi(z|\theta)$: Gọi là *Score Function* (cho biết hướng thay đổi tham số để tăng xác suất sinh ra mẫu z).
- $f(z_k)$: Đóng vai trò trọng số.



2.4. Áp dụng cho Phân phối Gaussian đa biến

Xét trường hợp phổ biến nhất: Phân phối chuẩn d -chiều $\mathcal{N}(\mu, \Sigma)$.

- Tham số $\theta = \langle \mu, \Sigma \rangle$.
- Log-density:

$$\log \pi(z|\theta) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log \det \Sigma - \frac{1}{2} (z - \mu)^\top \Sigma^{-1} (z - \mu)$$

Đạo hàm riêng (Search Gradients) cụ thể như sau:

$$\nabla_{\mu} \log \pi(z) = \Sigma^{-1} (z - \mu) \quad (4)$$

$$\nabla_{\Sigma} \log \pi(z) = \frac{1}{2} \Sigma^{-1} (z - \mu) (z - \mu)^\top \Sigma^{-1} - \frac{1}{2} \Sigma^{-1} \quad (5)$$



2.5. Tại sao Search Gradient thông thường thất bại?



2.5.1. Log-Likelihood

Xét phân phối chuẩn 1 chiều với $\theta = \langle \mu, \sigma \rangle$. Hàm mật độ xác suất:

$$\pi(z|\mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(z - \mu)^2}{2\sigma^2}\right)$$

Lấy Logarit tự nhiên (log) để đơn giản hóa việc tính đạo hàm:

$$\begin{aligned}\log \pi(z|\theta) &= \log\left((2\pi)^{-1/2}\right) + \log(\sigma^{-1}) + \log\left(e^{-\frac{(z-\mu)^2}{2\sigma^2}}\right) \\ &= \underbrace{-\frac{1}{2} \log(2\pi) - \log(\sigma)}_{\text{Hằng số } C} - \frac{1}{2}\sigma^{-2}(z - \mu)^2\end{aligned}$$

Hàm mục tiêu cần đạo hàm:

$$L = C - \log(\sigma) - \frac{1}{2\sigma^2}(z - \mu)^2$$



2.5.2. Biến đổi 1: Đạo hàm theo μ

Ta lấy đạo hàm riêng của L theo μ . Áp dụng quy tắc chuỗi (chain rule) cho cụm $(z - \mu)^2$:

$$\frac{\partial L}{\partial \mu} = \frac{\partial}{\partial \mu} \left[-\frac{1}{2\sigma^2} (z - \mu)^2 \right]$$

Đặt $u = z - \mu \Rightarrow \frac{du}{d\mu} = -1$. Ta có:

$$\begin{aligned} \frac{\partial L}{\partial \mu} &= -\frac{1}{2\sigma^2} \cdot \frac{d}{du}(u^2) \cdot \frac{du}{d\mu} \\ &= -\frac{1}{2\sigma^2} \cdot (2u) \cdot (-1) \\ &= \frac{u}{\sigma^2} = \frac{z - \mu}{\sigma^2} \end{aligned}$$

Kết quả 1: $\nabla_{\mu} \log \pi(z) = \frac{z - \mu}{\sigma^2}$



2.5.3. Biến đổi 2: Đạo hàm theo σ

Hàm mục tiêu: $L = C - \log(\sigma) - \frac{1}{2}(z - \mu)^2\sigma^{-2}$.

Đạo hàm từng thành phần theo σ :

- Đạo hàm của $-\log(\sigma)$ là: $-\frac{1}{\sigma}$
- Đạo hàm của σ^{-2} là: $-2\sigma^{-3}$

Thay vào biểu thức:

$$\begin{aligned}\frac{\partial L}{\partial \sigma} &= -\frac{1}{\sigma} - \frac{1}{2}(z - \mu)^2 \cdot (-2\sigma^{-3}) \\ &= -\frac{1}{\sigma} + \frac{(z - \mu)^2}{\sigma^3}\end{aligned}$$

Quy đồng mẫu số chung σ^3 :

$$\nabla_{\sigma} \log \pi(z) = \frac{-\sigma^2 + (z - \mu)^2}{\sigma^3} = \frac{(z - \mu)^2 - \sigma^2}{\sigma^3}$$



2.5.4. Tại sao Search Gradient thông thường thất bại?

Kết quả Gradient đối với σ :

$$\nabla_{\sigma} J \propto \frac{(z - \mu)^2 - \sigma^2}{\sigma^3}$$

Sự mất ổn định:

- 1 **Yêu cầu hội tụ:** Khi thuật toán tìm kiếm lời giải, σ cần giảm dần về 0 ($\sigma \rightarrow 0$).
- 2 **Nghịch lý toán học:** Mẫu số là σ^3 . Khi σ tiến về 0, giá trị $\frac{1}{\sigma^3}$ tiến ra vô cùng (∞).
- 3 **Hậu quả thực tế:** Gradient trở nên "bùng nổ". Bước cập nhật $\theta_{new} = \theta - \eta \nabla$ quá lớn khiến thuật toán văng xa khỏi cực trị.



2.5.4. Tại sao Search Gradient thông thường thất bại?

CHÈN GIF VÀO ĐÂY



2.6. Giải pháp: Natural Gradient

Thay vì dùng khoảng cách Euclidean, ta dùng khoảng cách giữa các phân phối xác suất (Kullback-Leibler Divergence) để kiểm soát bước nhảy.

Bài toán tối ưu ràng buộc (Trust Region):

$$\begin{aligned} \max_{\delta\theta} \quad & J(\theta + \delta\theta) \approx J(\theta) + \delta\theta^\top \nabla_\theta J \\ \text{s.t.} \quad & D_{KL}(\pi(\cdot|\theta + \delta\theta) || \pi(\cdot|\theta)) = \varepsilon \end{aligned} \tag{6}$$

Trong đó: $D_{KL} \approx \frac{1}{2} \delta\theta^\top \mathbf{F} \delta\theta$ (Khai triển Taylor bậc 2).

Giải thích ý nghĩa

- **Tại sao dùng dấu "="?** Hàm mục tiêu J được xấp xỉ tuyến tính (mặt phẳng nghiêng), nên điểm tối ưu luôn nằm ở **biên** của vùng tin cậy.



2.6.b. Tại sao F xuất hiện trong D_{KL} ?

Để tính ràng buộc, ta **khai triển Taylor bậc 2** của hàm $D_{KL}(\pi_\theta || \pi_{\theta+\delta\theta})$ quanh điểm $\delta\theta = 0$:

$$D_{KL} \approx \underbrace{D_{KL}(\theta || \theta)}_{=0} + \underbrace{\nabla D_{KL}|_{\delta\theta=0}^\top}_{=0} \delta\theta + \frac{1}{2} \delta\theta^\top \underbrace{\nabla^2 D_{KL}|_{\delta\theta=0}}_{\mathbf{F}} \delta\theta \quad (7)$$

Giải thích các thành phần:

- **Hạng tử bậc 0 ($= 0$):** Khoảng cách giữa hai phân phối giống hệt nhau luôn bằng 0.
- **Hạng tử bậc 1 ($= 0$):** Vì $D_{KL} \geq 0$ và đạt cực tiểu toàn cục tại $\delta\theta = 0$, nên đạo hàm bậc 1 (Gradient) tại đó phải bằng 0.
- **Hạng tử bậc 2 ($= F$):** Trong thống kê học, người ta chứng minh được rằng: *Ma trận Hessian (đạo hàm bậc 2) của KL-divergence chính là Ma trận thông tin Fisher (F).*

$$\Rightarrow D_{KL}(\pi_\theta || \pi_{\theta+\delta\theta}) \approx \frac{1}{2} \delta\theta^\top \mathbf{F} \delta\theta$$

2.7. Ma trận thông tin Fisher (F)

Giải bài toán tối ưu trên bằng nhân tử Lagrange, ta thu được hướng cập nhật tối ưu (Natural Gradient):

$$\tilde{\nabla}_{\theta} J = F^{-1} \nabla_{\theta} J \quad (8)$$

Với F là ma trận Fisher, được định nghĩa là hiệp phương sai của score function:

$$F = \mathbb{E} [\nabla_{\theta} \log \pi(z|\theta) \nabla_{\theta} \log \pi(z|\theta)^{\top}] \quad (9)$$

Ý nghĩa của F^{-1} :

- Nó đóng vai trò như một bộ "tiền điều kiện" (Pre-conditioner).
- Nó triệt tiêu ảnh hưởng của độ cong không gian tham số (ví dụ: nhân tử $1/\sigma^3$ sẽ bị F triệt tiêu), giúp bước cập nhật ổn định bất kể σ lớn hay nhỏ.



2.7. Ma trận Thông tin Fisher (F)

Chèn animation vào đây.



2.8. Tổng hợp: Thuật toán Canonical NES

Algorithm 1 Canonical Natural Evolution Strategies

```
1: Input:  $f, \theta_{init}$ , learning rate  $\eta$ 
2: repeat
3:   for  $k = 1 \dots \lambda$  do
4:     Draw  $z_k \sim \pi(\cdot|\theta)$ 
5:     Evaluate  $f(z_k)$ 
6:     Calculate gradients  $\nabla_{\theta} \log \pi(z_k|\theta)$ 
7:   end for
8:    $\nabla_{\theta} J \leftarrow \frac{1}{\lambda} \sum_{k=1}^{\lambda} f(z_k) \nabla_{\theta} \log \pi(z_k|\theta)$ 
9:    $F \leftarrow \frac{1}{\lambda} \sum_{k=1}^{\lambda} \nabla_{\theta} \log \pi(z_k|\theta) \nabla_{\theta} \log \pi(z_k|\theta)^{\top}$ 
10:   $\theta \leftarrow \theta + \eta F^{-1} \nabla_{\theta} J$  {Natural Update Step}
11: until stopping criterion met
```



Chương 3:

xNES: Exponential NES



3.0. Sinh mẫu Gaussian: Phân rã Cholesky

Vấn đề thực tế

Máy tính chỉ có thể sinh số ngẫu nhiên chuẩn tắc $z \sim \mathcal{N}(0, I)$ (đám mây hình cầu). Làm sao để tạo ra mẫu $x \sim \mathcal{N}(\mu, \Sigma)$ (đám mây hình elip nghiêng)?

Giải pháp: Phân rã Cholesky Mọi ma trận hiệp phương sai Σ (đối xứng, xác định dương) đều có thể phân rã thành:

$$\Sigma = AA^T$$

Trong đó A là ma trận tam giác dưới. Khi đó, mẫu x được sinh ra bằng biến đổi tuyến tính:

$$x = \mu + Az$$

Ý nghĩa:

- A đóng vai trò như "căn bậc hai" của ma trận Σ .
- Việc tối ưu hóa Σ trong thuật toán thực chất là tìm kiếm ma trận A để biến đổi hình dáng đám mây điểm sao cho khớp với địa hình hàm số.



3.. Khái niệm: Tính bất biến tuyến tính (Linear Invariance)

Định nghĩa

Một thuật toán có tính bất biến tuyến tính nếu hiệu suất của nó không thay đổi dưới bất kỳ phép biến đổi tuyến tính nào của không gian tìm kiếm (xoay, co giãn, lật hệ trục tọa độ).

Tại sao quan trọng?

- **Vấn đề Ill-conditioned:** Các bài toán thực tế thường có các chiều biến thiên không đều (ví dụ: x_1 thay đổi theo đơn vị mét, x_2 thay đổi theo milimet), tạo thành các "thung lũng" hẹp và dài.
- **Gradient thường:** Bị méo mó theo hệ trục, dẫn đến việc di chuyển zíc-zắc rất chậm (như hình minh họa slide trước).
- **Natural Gradient:** Có khả năng "chuẩn hóa" không gian cục bộ, biến thung lũng hẹp trở lại thành hình tròn, giúp thuật toán đi thẳng về đích bất kể đơn vị đo lường.



3.1. Tại sao cần xNES?

Hạn chế của các phiên bản trước:

- **Mất tính bất biến:** Việc tham số hóa bằng phân rã Cholesky làm mất tính bất biến đối với phép biến đổi tuyến tính, trừ khi bước nhảy cực nhỏ.
- **Hội tụ sớm:** Thường gặp khó khăn với các hàm phức tạp (như Rosenbrock) trong số chiều lớn.
- **Chi phí tính toán:** Việc tính nghịch đảo ma trận Fisher rất tốn kém ($O(d^3)$ đến $O(d^6)$).

Mục tiêu của xNES:

- 1 Khôi phục hoàn toàn tính bất biến tuyến tính.
- 2 Đảm bảo ma trận hiệp phương sai luôn xác định dương một cách tự nhiên.
- 3 Loại bỏ hoàn toàn việc tính ma trận Fisher (nhờ hệ tọa độ tự nhiên).



3.2. Tham số hóa hàm mũ

Vấn đề: Cập nhật cộng $C \leftarrow C + \delta C$ có thể làm ma trận C không còn xác định dương.

Giải pháp: Sử dụng ánh xạ mũ từ không gian ma trận đối xứng $\mathcal{S}_d$ sang đa tạp ma trận xác định dương $\mathcal{P}_d$.

$$\exp : \mathcal{S}_d \rightarrow \mathcal{P}_d, \quad M \mapsto \sum_{n=0}^{\infty} \frac{M^n}{n!}$$

Lợi ích:

- $\exp(M)$ luôn là ma trận xác định dương với mọi M đối xứng.
- Cho phép cập nhật trong không gian vector phẳng $\mathcal{S}_d$ mà không cần ràng buộc phức tạp.
- Tuy nhiên, tính ma trận Fisher trực tiếp vẫn tồn độ phức tạp rất lớn $\rightarrow$ Cần giải pháp thứ 2.



3.3. Hệ tọa độ tự nhiên (Natural Coordinates)

Thay vì dùng tọa độ toàn cục, tại mỗi bước lặp, ta chuyển sang hệ tọa độ cục bộ nơi phân phối hiện tại là chuẩn tắc $\mathcal{N}(0, I)$.

Gọi phân phối hiện tại là $\mathcal{N}(\mu, AA^T)$. Ta biểu diễn phân phối mới thông qua biến thiên (δ, M) :

$$\mu_{new} = \mu + A\delta \quad (10)$$

$$A_{new} = A \cdot \exp\left(\frac{1}{2}M\right) \quad (11)$$

Tính chất: Tại điểm gốc $(\delta = 0, M = 0)$, ma trận Fisher là ma trận đơn vị: $\mathbf{F} = \mathbf{I}$.

$$\Rightarrow \nabla_{nat} J = F^{-1} \nabla J = \nabla J$$

Gradient tự nhiên chính là Gradient thường trong hệ tọa độ này!



3.4. Log-Likelihood trong tọa độ tự nhiên

Với mẫu $x = \mu + Az$ (với $z \sim \mathcal{N}(0, I)$), mật độ xác suất trong hệ tọa độ (δ, M) là:

$$\log p(x|\delta, M) = -\frac{d}{2} \log(2\pi) - \text{tr}(\log A_{new}) - \frac{1}{2} \|A_{new}^{-1}(x - \mu_{new})\|^2$$

Thay thế các tham số từ phương trình (9) và (10), và xấp xỉ Taylor bậc 1 quanh 0:

$$\log p(z|\delta, M) \approx C - \underbrace{\frac{1}{2} \text{tr}(M)}_{\text{từ det}} - \underbrace{\frac{1}{2} \|(I - \frac{1}{2}M)(z - \delta)\|^2}_{\text{từ số mũ}}$$

Ta sẽ dùng công thức này để tính đạo hàm theo δ và M .



3.5. Dẫn xuất Gradient vị trí (G_δ)

Lấy đạo hàm log-likelihood theo δ tại $\delta = 0, M = 0$:

$$\begin{aligned}\nabla_\delta \log p &\approx \nabla_\delta \left(-\frac{1}{2} \|z - \delta\|^2 \right) \Big|_{\delta=0} \\ &= \nabla_\delta \left(-\frac{1}{2} (z^T z - 2z^T \delta + \delta^T \delta) \right) \Big|_{\delta=0} = z\end{aligned}$$

Áp dụng cho toàn bộ quần thể với trọng số (utilities) u_i :

$$G_\delta = \sum_{i=1}^n u_i \cdot z_i$$



3.6. Dẫn xuất Gradient ma trận (G_M)

Lấy đạo hàm log-likelihood theo M (bỏ qua các số hạng bậc cao):

$$\begin{aligned}\nabla_M \log p &\approx \nabla_M \left(-\frac{1}{2} \text{tr}(M) - \frac{1}{2} z^T (I - M) z \right) \\ &= -\frac{1}{2} I + \frac{1}{2} z z^T = \frac{1}{2} (z z^T - I)\end{aligned}$$

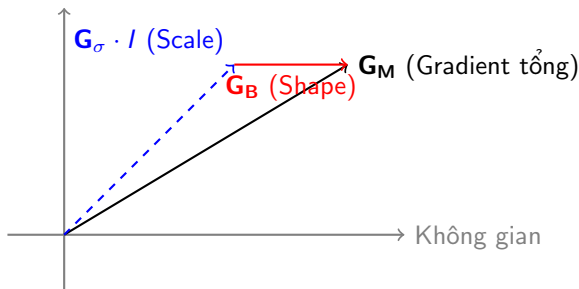
Tổng hợp trên quần thể (đưa hằng số $1/2$ ra ngoài):

$$G_M = \sum_{i=1}^n u_i \cdot (z_i z_i^T - I)$$



3.7.b. Minh họa

Việc tách G_M giúp kiểm soát riêng biệt độ lớn bước nhảy (Scale) và hướng xoay (Shape).



$$G_M = \text{Thay đổi kích thước} + \text{Biến dạng hình học}$$

1. Scale (G_σ):

$$\text{Trace} \neq 0$$

2. Shape (G_B):

$$\text{Trace} = 0$$

Phóng to/Thu nhỏ toàn bộ đám mây mà không đổi hình dạng.

Làm méo hình dạng (Elip hóa) nhưng giữ nguyên thể tích khối.



3.7. Phân rã tham số (Decomposition)

Để kiểm soát tốt hơn, xNES phân rã không gian cập nhật thành hai phần trực giao:

$$\mathbb{R}^d \times \mathcal{S}_d = \underbrace{\mathbb{R}^d}_{(\mu)} \times \underbrace{\mathcal{S}^d}_{(\sigma)} \times \underbrace{\mathcal{S}_d^\perp}_{(B)}$$

1. **Scale Update** (σ): Hướng đẳng hướng (Isotropic), thay đổi kích thước bước nhảy.

$$G_\sigma = \frac{\text{tr}(G_M)}{d}$$

2. **Shape Update** (B): Hướng lệch (Anisotropic), thay đổi hình dạng phân phối (với $\text{trace} = 0$).

$$G_B = G_M - G_\sigma \cdot I$$

Điều này cho phép dùng *learning rate* riêng biệt (η_σ, η_B) cho kích thước và hình dáng.



3.8. Quy tắc cập nhật (Update Rules)

Kết hợp các gradient đã dẫn xuất với phép ánh xạ mũ:

1. Cập nhật tâm (μ):

$$\mu_{new} = \mu + \eta_{\mu} \cdot \sigma B \cdot G_{\delta}$$

2. Cập nhật quy mô (σ) - Exponential:

$$\sigma_{new} = \sigma \cdot \exp\left(\frac{\eta_{\sigma}}{2} G_{\sigma}\right)$$

3. Cập nhật ma trận biến đổi (B) - Exponential Map:

$$B_{new} = B \cdot \exp\left(\frac{\eta_B}{2} G_B\right)$$

*Lưu ý: $A = \sigma B$.



3.9. Tổng kết thuật toán xNES

-
- 1: **Init:** $\mu, \sigma, B = I$
 - 2: **Compute utilities:** u_i based on rank
 - 3: **loop**
 - 4: Sample $z_i \sim \mathcal{N}(0, I)$ and map $x_i = \mu + \sigma B z_i$
 - 5: Evaluate $f(x_i)$ and sort to match utilities
 - 6: **Compute Natural Gradients (in local coords):**
 - 7: $G_\delta = \sum u_i z_i$
 - 8: $G_M = \sum u_i (z_i z_i^T - I)$
 - 9: $G_\sigma = \text{tr}(G_M)/d, \quad G_B = G_M - G_\sigma I$
 - 10: **Update Parameters:**
 - 11: $\mu \leftarrow \mu + \eta_\mu \sigma B G_\delta$
 - 12: $\sigma \leftarrow \sigma \cdot \exp(\eta_\sigma G_\sigma / 2)$
 - 13: $B \leftarrow B \cdot \exp(\eta_B G_B / 2)$
 - 14: **end loop**
-



3.10. XNES vs NES

chèn demo vào đây



Chương 4:

So sánh & Thực nghiệm



4.1. NES vs CMA-ES

Cả hai đều là tiêu chuẩn vàng trong tối ưu hóa tiến hóa Gaussian.

- **Giống nhau:** Dùng phân phối Gaussian, dùng rank-based weights.
- **Khác nhau:**
 - **CMA-ES:** Dựa trên Heuristics (Evolution paths). Cực kỳ mạnh trong thực tế nhưng thiếu nền tảng toán học thống nhất.
 - **NES:** Dựa trên Natural Gradient (Fisher). Có cơ sở lý thuyết vững chắc, dễ mở rộng sang các phân phối khác.

Kết luận

CMA-ES là lựa chọn thực dụng; NES là lựa chọn cho nghiên cứu sâu và các bài toán cần tính bất biến tuyệt đối.



4.2. Thực nghiệm

Bài toán: Tối ưu hóa hàm **Rastrigin** với $d = 2$.

$$f(\mathbf{x}) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$$

- **Đặc điểm:** Non-convex, cực kỳ nhiều bẫy local minima.
- **Mục tiêu:** Tìm Global Minimum tại $\mathbf{x} = \mathbf{0}$, $f(\mathbf{x}) = 0$.



4.3. Kết quả với $d = 2$

CHÈN GIF VÀO ĐÂY



4.3. Kết quả với $d = 2$

CHÈN GIF VÀO ĐÂY



4.3. Kết luận

- ➊ NES cung cấp nền tảng toán học vững chắc cho tiến hóa thông qua Hình học thông tin.
- ➋ Natural Gradient là chìa khóa để đạt được sự ổn định và tính bất biến.

Cảm ơn Thầy và các bạn!

